

Análisis - Ejercicio 5

Enmanuel García 26846

Lourdes Lemus 26305

1/10/26

1. Descripción del problema

El programa consiste en un juego de tablero cuadriculado que simula la supervivencia de estudiantes durante una semana de exámenes parciales. En el tablero, que representa el campus universitario, se encuentran diferentes tipos de piezas que interactúan entre sí.

Las piezas principales son estudiantes, catedráticos y recursos u obstáculos. Aunque todas comparten características generales, como un identificador, nombre, posición dentro del tablero y puntos de estabilidad, cada tipo de pieza posee características y comportamientos particulares.

El sistema deberá crear inicialmente un tablero con al menos diez piezas de distintos tipos. Posteriormente, mediante un menú de consola, el usuario podrá consultar las piezas existentes, buscar una pieza específica y ordenar las piezas de acuerdo con sus puntos de estabilidad.

Debido a que existen diferentes objetos que comparten características generales pero presentan comportamientos distintos, el programa deberá aprovechar los conceptos de herencia y polimorfismo de la programación orientada a objetos. Además, deberá mantener una adecuada separación de responsabilidades mediante la aplicación del patrón MVC.

2. Requisitos funcionales

1. Carga inicial de piezas: Al iniciar el programa se deberá realizar una carga inicial del tablero con al menos diez piezas de diferentes tipos.
2. Almacenamiento de diferentes tipos de piezas: El sistema deberá permitir almacenar estudiantes, catedráticos y recursos u obstáculos dentro de una misma colección de piezas.
3. Listado de piezas: El usuario deberá poder visualizar todas las piezas registradas en el campus, incluyendo su estado y posición actual.
4. Búsqueda por ID: El usuario deberá poder buscar una pieza utilizando su identificador.
5. Búsqueda por nombre: El usuario deberá poder buscar una pieza utilizando su nombre.
6. Consulta de estadísticas: Cuando se encuentre una pieza mediante una búsqueda, el sistema deberá mostrar sus estadísticas correspondientes.
7. Ordenamiento por estabilidad: El usuario deberá poder ordenar las piezas según sus puntos de estabilidad. Para realizar este ordenamiento se utilizará la interfaz Comparable.
8. Comportamiento polimórfico: Cada tipo de pieza deberá implementar su propio comportamiento al ejecutar un turno mediante el método `ejecutarTurno()`.

9. Menú de consola: El sistema deberá proporcionar un menú de consola mediante el cual el usuario pueda acceder a las funcionalidades requeridas.

10. Uso de la información inicial: La información presentada por el programa deberá corresponder al estado registrado durante la carga inicial.

3. Identificación de clases

- 1) **Pieza:** Se creará una clase abstracta llamada Pieza, debido a que estudiantes, catedráticos y recursos u obstáculos comparten un conjunto de características. La clase concentrará los atributos comunes de todas las piezas para evitar duplicación de información en las subclases. Esta será abstracta porque no se necesita crear directamente objetos que sean únicamente "piezas"; los objetos creados pertenecerán a alguna de sus subclases.

Los atributos identificados son:

- id: identificador de la pieza.
- nombre: nombre de la pieza.
- x: posición horizontal dentro del tablero.
- y: posición vertical dentro del tablero.
- puntosEstabilidad: valor entero que representa el nivel de estabilidad de la pieza.

Pieza declarará el método abstracto ejecutarTurno(), obligando a las subclases a proporcionar su propia implementación. La clase también implementará Comparable<Pieza> para permitir que diferentes tipos de piezas puedan ordenarse según sus puntos de estabilidad.

- 2) **Estudiante:** La clase Estudiante será una especialización de Pieza. Además de las características heredadas, tendrá atributos relacionados con el estado particular del estudiante. El enunciado indica que los estudiantes se desplazan por el tablero buscando café y poseen puntos relacionados con energía o estrés.

Por lo tanto, sus atributos serán:

- energia: representa la energía disponible del estudiante.
- estres: representa el nivel de estrés acumulado.

Estudiante sobrescribirá ejecutarTurno() para proporcionar el comportamiento correspondiente a este tipo de pieza.

- 3) **Catedrático:** La clase Catedratico heredarán de Pieza y representará a los catedráticos presentes en el campus. El enunciado establece que los catedráticos poseen un área de efecto en la cual pueden asignar proyectos sorpresa.

Por ello se propone agregar:

- areaEfecto: representa el alcance que posee el catedrático dentro del tablero.

La clase sobrescribirá `ejecutarTurno()` para implementar un comportamiento asociado con la asignación de proyectos sorpresa.

- 4) **RecursoObstaculo:** La clase `RecursoObstaculo` heredará de `Pieza` y permitirá representar los diferentes elementos disponibles dentro del campus, como máquinas de café, enchufes, impresoras, mesas de trabajo y otros recursos u obstáculos.

Se incorporará:

- `durabilidad`: representa el estado actual del recurso u obstáculo.
- `usosDisponibles`: representa la cantidad de veces que todavía puede utilizarse.

La clase sobrescribirá `ejecutarTurno()` para proporcionar un comportamiento propio de este tipo de pieza.

- 5) **Campus:** La clase `Campus` tendrá la responsabilidad de administrar las piezas presentes en el tablero. Mantendrá una colección de tipo `ArrayList<Pieza>`. El uso del tipo `Pieza` permitirá almacenar dentro de una misma colección objetos de diferentes subclases, como `Estudiante`, `Catedratico` y `RecursoObstaculo`.

Entre sus responsabilidades se encontrarán:

- Realizar la carga inicial de las piezas.
- Almacenar las piezas.
- Proporcionar acceso al listado de piezas.
- Buscar piezas.
- Ordenar las piezas según sus puntos de estabilidad.

La utilización de una colección de `Pieza` permitirá aprovechar el polimorfismo y evitar la creación de colecciones independientes para cada tipo de objeto.

- 6) **JuegoView:** Será responsable de la interacción con el usuario mediante consola. Se encargará de mostrar el menú, solicitar la información necesaria y presentar los resultados obtenidos. Esta clase no realizará directamente operaciones relacionadas con la lógica del modelo.

- 7) **JuegoController:** Será responsable de coordinar la comunicación entre el modelo y la vista. Recibirá las opciones seleccionadas por el usuario mediante `JuegoView` y solicitará a `Campus` la ejecución de las operaciones correspondientes.

4. Herencia

La herencia se utilizará debido a que `Estudiante`, `Catedratico` y `RecursoObstaculo` comparten características comunes. Estas características se definirán una sola vez dentro de la superclase `Pieza`. Posteriormente, las subclases heredarán dicha información y agregarán únicamente los atributos y comportamientos que les correspondan.

La jerarquía será:

Pieza:

- Estudiante
- Catedratico
- RecursoObstaculo

Con esta estructura se reduce la duplicación de código y se establece una relación generalización y especialización entre las clases.

5. Polimorfismo

El polimorfismo permitirá manipular objetos de diferentes subclases mediante referencias de tipo Pieza. El programa podrá mantener una colección `ArrayList<Pieza>` que contenga estudiantes, catedráticos y recursos u obstáculos. La clase padre Pieza declarará el método `ejecutarTurno()`, mientras que cada subclase realizará un `override` del método para proporcionar su propia implementación.

De esta manera, cuando `ejecutarTurno()` sea invocado mediante una referencia de tipo Pieza, el comportamiento ejecutado dependerá del tipo real del objeto. Esto permite agregar diferentes tipos de piezas a una misma estructura sin que el código que trabaja con la colección necesite conocer necesariamente la subclase concreta de cada objeto.

6. Comparable

La clase Pieza implementará la interfaz `Comparable<Pieza>`.

El criterio de comparación serán los puntos de estabilidad de cada pieza. De esta forma, será posible ordenar una colección que contenga diferentes tipos de piezas sin importar si cada objeto es un Estudiante, Catedratico o RecursoObstaculo.

El método `compareTo()` comparará los puntos de estabilidad del objeto actual con los puntos de estabilidad de otra pieza. El ordenamiento se realizará de menor a mayor estabilidad, permitiendo identificar primero las piezas que posean los valores de estabilidad más bajos.

7. Overloading

Se utilizará `overloading` en las operaciones de búsqueda de piezas.

El sistema necesita permitir búsquedas tanto por ID como por nombre. Debido a que ambas operaciones representan conceptualmente la acción de buscar una pieza, se utilizará el mismo nombre de método con diferentes parámetros.

Se usarán las siguientes operaciones:

- `buscarPieza(int id)`
- `buscarPieza(String nombre)`

De esta manera se aplica sobrecarga de métodos manteniendo una interfaz coherente para las operaciones de búsqueda.

8. MVC

El programa utilizará el patrón Modelo-Vista-Controlador para separar las responsabilidades del sistema.

Modelo: Estará compuesto por las clases Pieza, Estudiante, Catedratico, RecursoObstaculo y Campus. Estas clases representarán los datos y la lógica relacionada con las piezas del juego.

Vista: La clase JuegoView será responsable de la interacción mediante consola. Mostrará el menú, solicitará información al usuario y presentará los resultados. La vista no será responsable de realizar búsquedas, ordenamientos ni modificar directamente la colección del modelo.

Controlador: La clase JuegoController funcionará como intermediario entre el modelo y la vista. Recibirá las opciones seleccionadas por el usuario mediante la vista y solicitará al modelo las operaciones necesarias.

Esta separación permitirá evitar que una sola clase concentre la lógica del programa, la administración de los datos y la interacción con el usuario.

9. Buenas prácticas

El diseño utilizará encapsulación mediante atributos privados y métodos de acceso cuando sean necesarios.

La clase Pieza sobrescribirá el método toString() para proporcionar una representación textual de sus características principales. Las subclases podrán extender esta representación agregando sus atributos particulares. También se implementará equals() cuando sea necesario comparar lógicamente dos piezas. Se propone utilizar el ID como identificador único de una pieza.

Los métodos y atributos utilizarán modificadores de visibilidad de acuerdo con su responsabilidad, evitando exponer directamente el estado interno de los objetos.

10. UML